1章
必要十分なマイクロサービス

大変な騒ぎだったな。一気に収拾がつかなくなった。

映画『俺たちニュースキャスター』

マイクロサービスの扱い方に飛び込む前に、マイクロサービスアーキテクチャとは何かについて共通の理解を持つことが重要だ。ここでは、日常的に目にする一般的な誤解や見落としがちなニュアンスを取り上げていく。本書の残りの部分を最大限に活用するには、マイクロサービスアーキテクチャについての確固たる知識基盤が必要だ。そのため、この章では、マイクロサービスアーキテクチャについて説明し、マイクロサービスがどのように発展してきたのかをざっと確認し（当然ながらモノリスについても見ていく）、マイクロサービスを扱う上でのいくつかの利点と課題を考察していく。

1.1　マイクロサービスとは

マイクロサービスとは、ビジネスドメインに基づいてモデル化された、独立してデプロイ可能なサービスだ。サービス同士はネットワークを介して相互に通信してシステムを形成する。このアーキテクチャの選択は、あなたが直面するであろう問題を解決するための多くの選択肢を提供する。マイクロサービスアーキテクチャとは、複数の連携しあうマイクロサービスによって成り立っているアーキテクチャということになる。

マイクロサービスは、サービス指向アーキテクチャ（Service-Oriented Architecture：SOA）の**一種**ながら、サービスの境界をどのように引くべきかについて確固とした考え方があり、独立してデプロイ可能であることが重要なアーキテクチャだ。そして、マイクロサービスは、何らかの技術に依存しないという利点も備えている。

技術的な観点から見ると、マイクロサービスとは、カプセル化されたビジネス機能を1つ以上のネットワークエンドポイントを介して公開するものだ。それらはネットワークを介して相互に通信しあい、分散システムを形成する。また、マイクロサービスはデータの保存や検索をカプセル化し、適切に定義されたインターフェイスを介してデータを公開する。つまり、データベースはサービス境界の内側に隠されることになる。

それぞれについて、解き明かすべきことが多くある。ここからは、これらの考えをもう少し掘り下げていこう。

1.1.1　独立デプロイ可能性

独立デプロイ可能性とは、あるマイクロサービスに対する変更を、他のサービスに影響を与えることなく本番環境へデプロイできるという考え方だ。単にそれが行えるという以上に、システムにおけるデプロイが実際にそのように管理されているということが重要だ。それはリリースの大半で実践されるべき規律でなくてはならない。この考えは単純だが、行うのはとても難しい。

本書から得られるものが1つだけだとしたら、それは「マイクロサービスに独立デプロイ可能性の概念を確実に取り入れよう」ということだ。1つのマイクロサービスへの変更を、他の何かをデプロイすることなく本番環境にリリースする習慣を身につけよう。そうすれば、多くの良いことがついてくるはずだ。

独立デプロイ可能性を保証するには、サービス同士が**疎結合**である必要がある。つまり、他を変更することなく、あるサービスを変更できなくてはならない。そのためには、サービス間に明確で安定した契約[†1]が必要となる。これを難しくするような実装の選択も存在する。データベースを共有することなどは、特に問題となる。安定したインターフェイスを持つ疎結合なサービスを希求することは、そもそもサービス境界をどう見つけるかということについて考える指針となる。

[†1] 訳注：サービスがサービスの利用者に対して保証するインターフェイス（送受信するメッセージやエンドポイントなど）に関する約束事。

1.1.2　ビジネスドメインに基づくモデル化

プロセスの境界を越えて変更を加えるのはコストがかかる。あるフィーチャー†2を公開するために 2つのサービスに変更を加え、その2つの変更のデプロイをオーケストレーションする必要があるとする。この場合には、単一のサービス (あるいはモノリス) 内で同じ変更を行うよりも多くの作業が必要になる。したがって、私たちは、サービスをまたいだ変更をできる限り低い頻度で済ます方法を見つけたい。

[†2] 訳注：ソフトウェアやシステムの持つ機能や特徴のこと。

前著『マイクロサービスアーキテクチャ』[25]に倣って、ある概念を本書で説明する際に実世界のストーリーを共有できない場合、架空のドメインと会社を用いる。それは、Music Corpという会社だ。同社は、いまだにCD販売に完全に集中しているにもかかわらず、なぜかビジネスを継続している大規模な多国籍企業だ。

私たちは、Music Corpを21世紀へと移行させることに決めた。現在は、そのために既存のシステムアーキテクチャを評価しているところだ。図1-1に示すように、Music Corpのシステムはシンプルな3層構造になっている。Webベースのユーザーインターフェイス、モノリシックなバックエンドの形をしたビジネスロジック層、そして昔ながらのデータベースによるデータストレージ。そして、一般によく見られるように、これらの層はそれぞれ別のチームが所有している。

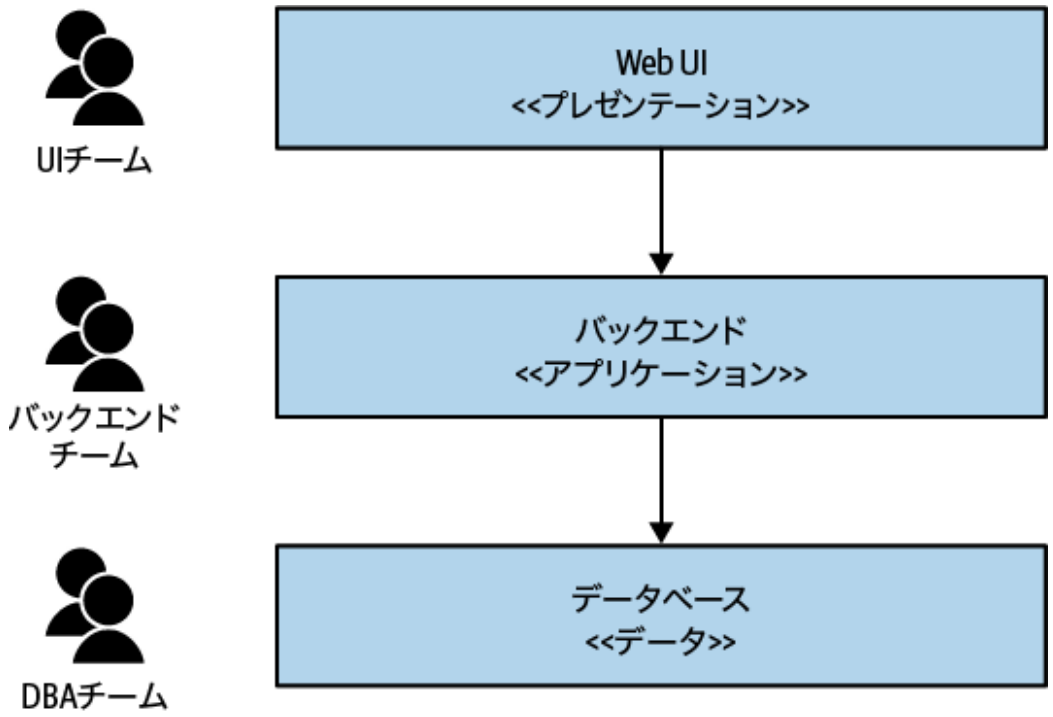

図1-1　Music Corpのシステムは伝統的な3層アーキテクチャをしている

今、顧客がお気に入りの音楽ジャンルを指定できるようにするという単純な変更を行いたいとしよう。この変更では、ジャンル選択のUIを表示するようにユーザーインターフェイスを変更し、UIにジャンルを表示して値を変更できるようにバックエンドサービスを変更し、その変更を受け入れるようデータベースを変更する必要がある。図1-2に示すように、これらの変更は各チームで管理する必要があり、正しい順序でデプロイする必要がある。

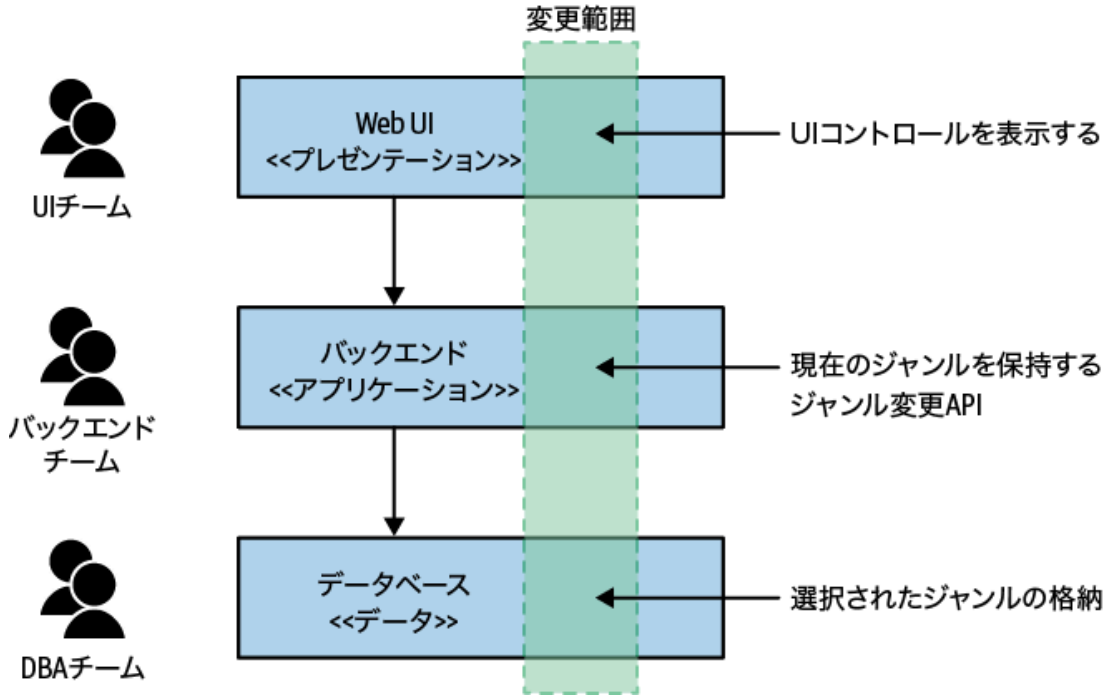

図1-2　3層を横断して行う変更はやや複雑になる

このアーキテクチャが悪いというわけではない。全てのアーキテクチャは、最終的にはいくつかの目標に基づいて最適化されるものだ。3層構造のアーキテクチャが非常に一般的なのは、それが普遍的で誰もが聞いたことがあるからだ。おそらくどこかで見たことがある一般的なアーキテクチャを選ぶというのが、しばしば、このパターンが選ばれる理由の1つになっている。しかし、このアーキテクチャが繰り返し選ばれる最大の理由は、それがチームの編成方法に基づいている点にあると私は考えている。

ここで、有名なコンウェイの法則を引用しよう。

> *システムを設計する組織は、その構造をそっくりまねた構造の設計を生み出してしまう。*
>
> *Melvin Conway “How Do Committees Invent?”*

3層アーキテクチャは、これが実際に行われている良い例だ。これまでのIT組織では、データベース管理者は他のデータベース管理者とチームを組み、Java開発者は他のJava開発者とチームを組み、フロントエンド開発者（最近ではJavaScriptやネイティブモバイルアプリケーションの開発など特異な知識に精通している）は別のチームに所属していた。私たちは人員をコアコンピテンシーに基づいて編成していたため、それに合わせたIT資産を作成してきた。

これこそ、3層アーキテクチャが普及した理由だ。これ自体は悪いことではない。3層アーキテクチャは、私たちが伝統的に人々をどうグループ化してきたかという力学や経験に基づいて最適化されているだけだ。しかし、力学は変化し、ソフトウェアに求めるものも変わった。私たちは今では、仕事の受け渡しやサイロを減らすために、職能横断型のチームを編成している。そして、ソフトウェアをこれまで以上に素早く出荷したいと考えている。そのため、私たちはチームをどのように編成するか、そしてシステムをどのように分割するかという点で、これまでとは異なる選択を推し進めていっている。

機能の変更とは、主にビジネス機能の変更を指す。しかし、図1-1では、ビジネス機能が実質的に3層全てに分散しているため、機能変更が層をまたいで行われる可能性が高くなっている。これは、このアーキテクチャは確かに技術についての凝集度は高いものの、ビジネス機能の凝集度については低いということを示している。変更を容易にしたいのであれば、技術ではなく、ビジネス機能のまとまりに基づいてコードをグループ化する必要がある。各サービスは、

最終的にはこれらの3つの層の混合物を含むかもしれないし、そうならないかもしれない。それはそれぞれのサービスの実装の課題となる。

前述の3層アーキテクチャを、図1-3に示す、他に代替できそうなアーキテクチャと比較してみよう。代替アーキテクチャには、専用の顧客サービスがあり、顧客が情報を更新できるようにUIを公開しており、顧客の状態もこのサービス内に保存されている。好きなジャンルの選択は特定の顧客に関連付けられているので、今回の変更はより局所的なものとなる。図1-3では、利用可能なジャンルのリストはカタログサービスから取得されている。カタログサービスは、おそらく既に導入されているものだろう。また、図中には、お気に入りのジャンル情報にアクセスする新しいレコメンデーションサービスもある。レコメンデーションサービスは、後続のリリースで容易にこの情報にアクセスして開発を行えるだろう。

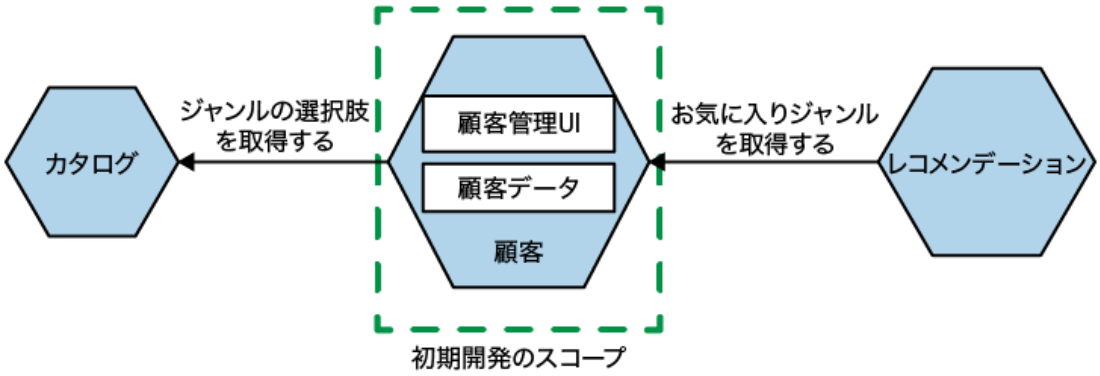

図1-3　顧客サービスを持つことで、お気に入りジャンルの記録がずっと容易になる

この場合、顧客サービスは薄くスライスした3層アーキテクチャをカプセル化したものとなる。顧客サービスは若干のUI、若干のアプリケーションロジック、若干のデータストレージを持つものの、これらの層は全て1つのサービス内にカプセル化されている。

ビジネスドメインがシステムアーキテクチャを推進する主な力となり、それによって、うまくいけば変更が容易になり、ビジネスドメインを中心にチームを編成することが容易になる。これはとても重要なことなので、ドメインに基づいてソフトウェアをモデリングする考え方については、この章の終わりに改めて触れることにする。そこでは、マイクロサービスアーキテクチャについての考え方を形づくる、ドメイン駆動設計周辺の考え方をいくつか紹介する。

1.1.3　自分たちのデータを所有する

おそらく誰もが一番苦労させられていることに、マイクロサービスはデータベースを共有すべきでないという考え方がある。あるサービスが他のサービスの保持するデータにアクセスしたい場合には、必要なデータをそのサービスに要求しなければならない。こうすることで、サービスは何を共有して何を隠すかを決める能力を手にできる。また、さまざまな理由で変更される可能性のある内部実装を、より安定したパブリックな契約へとマッピングでき、それによって安定したサービスインターフェイスを確保できる。独立デプロイ可能性を望むなら、サービス間で安定したインターフェイスを持つことは不可欠だ。サービスの公開インターフェイスが変わり続けると、それは他のサービスにも変更を要する波及効果をもたらすことになる。

本当に必要でない限りは、データベースを共有してはならない。そして、それを避けるためにできる限りのことをしよう。私の考えるところ、独立デプロイ可能性を実現しようとしているなら、データベースの共有は最悪の事項の一つだ。

前の節で説明したように、ビジネス機能をまるごとスライスしたもの、すなわちUIとアプリケーションロジック、データストレージを適切にカプセル化したものをサービスとしたい。ビジネス機能を変更する際に必要な労力を減らしたいからだ。データと振る舞いをカプセル化することで、ビジネス機能の凝集度を高められる。また、サービスを支えるデータベースを隠すことで、結合も確実に減らせる。結合と凝集については、後ほどすぐに改めて説明する。

特に、巨大なデータベースを扱う必要がある既存のモノリシックシステムを持っている場合には、これを理解するのは難しいかもしれない。幸いなことに4章はモノリシックデータベースからの脱却に完全に特化した章となっている。

1.1.4　マイクロサービスがもたらす利点

マイクロサービスの利点は多岐にわたる。デプロイを他と独立させることは、システムの規模と堅牢性を向上させるための新しいモデルを切り開き、技術を組み合わせて調整することを可能にする。各サービスの作業は並行して行えるため、より多くの開発者がお互いの邪魔をすることなく問題に取り組める。また、開発者はシステムの一部だけに注意を向けられるため、自分が担当している部分を理解するのが容易になる。さらに、プロセスの分離によって多様な技術の選択が可能になり、異なるプログラミング言語、プログラミングスタイル、デプロイメントプラットフォーム、データベースから、適切な組み合わせを見つけられる。

何よりも、マイクロサービスアーキテクチャは柔軟性を与えてくれる。マイクロサービスアーキテクチャは、将来的に問題を解決する方法について、より多くの選択肢を私たちに提供してくれる。

ただし、これらの利点のいずれもが簡単に手に入るものではないことに注意しなくてはならない。システムを分解する方法はいろいろある。本質的に何を達成しようとしているかによって、私たちはこの分解をさまざまな方向に進められる。したがって、マイクロサービスアーキテクチャから何を得ようとしているのかを理解することが重要になる。

1.1.5　マイクロサービスが作り出す問題

サービス指向アーキテクチャが台頭してきた背景には、コンピューターが安価になり、より多くのコンピューターを入手できるようになったことがある。システムを単一の巨大なメインフレームにデプロイするよりも、複数の安価なマシンを利用する方が理にかなうようになった。サービス指向アーキテクチャは、複数のマシンにまたがるアプリケーションをどのように構築するのが最善かを考えようとする試みだった。そうした試みに共通する主要な課題の1つは、コンピューターが互いに話をする方法、つまりネットワークだ。

ネットワーク越しのコンピューター間通信は、瞬時には行われない[†3]。これは、レイテンシー、とりわけローカル内の処理をはるかに上回るレイテンシーを気にする必要があることを意味する。レイテンシーが変動し、システムの動作を予測不可能にすることがあるのを考えると、事態はさらに悪化する。また、ネットワーク障害によってパケットが失われたり、ネットワークケーブルが切断されたりするという事象にも対処する必要がある。

[†3] これは物理学と関係があるようだ。

これらの課題は、単一プロセスのモノリスでは比較的単純に行えるトランザクションのような処理をはるかに困難にする。実のところ、システムが複雑になってくると、トランザクションとそれがもたらす安全性を捨てて、他の種類のテクニック（残念ながら、大きく異なるトレードオフがある）へと切り替えなければならなくなる可能性が高い。

ネットワーク呼び出しに失敗する可能性が常にあるという事実は頭痛の種になる。通信中のサービスが何らかの理由でオフラインになったり、奇妙な動作を始めたりといった事象が起きる可能性もある。さらには、複数のマシン間で一貫したデータビューを保つ方法も模索し始める必要がある。

もちろん、マイクロサービスに適した新しい技術も豊富に存在している。しかし、そうした新しい技術は、使い方を誤ると、より興味深く、より高くつく間違いを、より速く生む可能性がある。正直なところ、マイクロサービスは、良い部分を除けば、とんでもないアイデアのように思える。

私たちが「モノリス」として分類しているシステムの大半が、実質的には分散システムでもあることは注目に値する。単一プロセスのアプリケーションは、別のマシン上で動作するであろうデータベースからデータを読み取り、Webブラウザにデータを表示する。この場合は、少なくとも3台のコンピューターが混在しており、ネットワーク越しに通信し合うことを意味する。マイクロサービスアーキテクチャとの違いは、システムがどの程度分散されているかということだ。より多くのコンピューターが混在し、より多くネットワークを介して通信するようになると、分散システムに関連した厄介な問題にぶつかる可能性が高くなる。ここで簡単に説明した問題は、最初は現れないかもしれない。しかし、時間がたち、システムが成長するにつれて、全てではないにしても、ほとんどの問題にぶつかることになるだろう。

昔の同僚であり、友人であり、マイクロサービスの専門家でもあるJames Lewisは「マイクロサービスは選択肢を買い与える」と言っていた。この「マイクロサービスは**選択肢**を**買い与える**」という言葉は意図的に使われている。マイクロサービスにはコストがあり、そのコストが手にしたい選択肢に見合うものかどうかを判断しなければならない。このトピックについては、2章で詳しく説明する。

1.1.6　ユーザーインターフェイス

マイクロサービスを導入する際に、サーバーサイドにのみ焦点を当てて、ユーザーインターフェイスを単一のモノリシックなレイヤーのままで放っているケースをよく目にする。新しいフィーチャーをより迅速にデプロイしやすくするアーキテクチャを望むのであれば、UIをモノリスにしておくのは大きな誤りだろう。ユーザーインターフェイスもまた分解すべき対象だ。これについては3章の中で説明する。

1.1.7　技術

新しく輝かしいマイクロサービスアーキテクチャに合わせて、新しい技術を大量に取り入れたいという誘惑に駆られてしまうこともあるだろう。しかし、その誘惑に陥ってはいけない。新しい技術を採用するにはコストがかかる。うまくいけば、それだけの価値はある（もちろん、適切な技術を選択していればだ）。しかし、マイクロサービスアーキテクチャを最初に導入する際は、ただそれを行うだけで十分だ。

マイクロサービスアーキテクチャを適切に進化させ、管理する方法を見つけるには、分散システムに関する多くの課題に取り組む必要がある。そうした問題に遭遇したときには、自分が慣れ親しんだ技術スタックを利用して頭の中を整理し、既存の技術を変更することで問題に対処できるかを検討する方がはるかに有益だと私は考えている。

既に触れたように、マイクロサービスは基本的には技術に依存しない。ネットワークを介してサービスが相互に通信できる限り、なんでも利用可能だ。これは大きな利点となる。必要に応じて技術スタックを組み合わせることが可能となるからだ。

Kubernetes、Docker、コンテナ、パブリッククラウドを使う必要はないし、GoやRustなどでコーディングする必要もない。実際、マイクロサービスアーキテクチャに関しては、プログラミング言語の選択はさして重要ではない。もしあなたがPHPを一番よく知っているのであれば、PHPでサービスを構築しよう[†4]。残念ながら、特定のツールを使って仕事をしている人々を軽くみるような、一部の技術スタックに対する通ぶった態度というものがあまりにも多く横行している[†5]。そんなものに加担してはいけない。自分にとってうまくいくアプローチを選択し、直面した問題に対処するために物事を変えていこう。

[†4] PHPでサービスを構築するならLorna Jane Mitchell著 “PHP Web Services: APIs for the Modern Web”（O'Reilly）[23]をお勧めする。

[†5] Aurynn Shawのブログ記事 “Contempt Culture（軽蔑の文化）”（http://bit.ly/2oeICgL）を読んだ後、私は過去にさまざまな技術、そしてその周囲のコミュニティに対して軽蔑するような態度を示していたという罪を自覚した。

1.1.8　サイズ

「マイクロサービスはどのくらいの大きさにすべきか？」というのは、おそらく最もよくある質問だ。名前の中に「マイクロ」という単語が入っていることを考えると、これは当然に思える。しかし、マイクロサービスをアーキテクチャの一種として機能させているのは何なのか、ということを考えると、サイズという概念は実は最も面白みのないものの一つとなる。

サイズはどうやって測るのだろう？　コードの行数？　それはあまり意味がない。Javaでは25行のコードが必要なものが、Clojureでは10行で書けるかもしれない。ClojureがJavaより優れているとか劣っているとか言いたいわけではない。いくつかの言語は他の言語よりも表現力が高い、というだけだ。

マイクロサービスにおける「サイズ」の捉え方で私の考え方と最も近いのは、マイクロサービスの専門家であるChris Richardsonがかつて言っていたものだ。それは、マイクロサービスの目標は「可能な限り小さなインターフェイス」を持つことだという考えだ。これは、この後で説明する情報隠蔽の考えと一致しているが、実際には後付けの考えだ。なぜなら、最初にこれらのことを話していた際に私たちが主な焦点としていたのは、少なくとも当初はサービスを容易に交換できるようにすることだったからだ。

結局のところ、「サイズ」という概念は文脈に強く依存する。あるシステムに15年間携わってきた人であれば、そのシステムがたとえ10万行のコードで書かれたものであっても、分かりやすいと感じているだろう。しかし、そのプロジェクトに参加したばかりの人にとっては、それは大きすぎると感じるに違いない。同様に、マイクロサービスの移行に着手したばかりで、おそらく10個に満たないマイクロサービスしか持たない会社に尋ねた場合と、何年も前からマイ

クロサービスが当たり前で、今では何百個もマイクロサービスを持つ会社に尋ねた場合では、その2社が同じ規模の会社だったとしても、異なる答えが返ってくるだろう。

私はサイズのことは気にしないよう勧めている。最初にマイクロサービス化を始めるときには、2つの大切なポイントに焦点を当てる方がはるかに重要だ。第一に自分たちはどれくらいの数のマイクロサービスを扱えるだろうか？　サービスの数が増えれば増えるほど、システムの複雑さが増し、それに対応するために新しいスキルを習得しなければならなくなる（そして、おそらく新しい技術を採用しなければならないだろう）。そうした理由から、マイクロサービスアーキテクチャへの段階的な移行を強く推奨している。第二に、マイクロサービスの境界線をどのように定義すれば、全てがぐちゃぐちゃに結びついてしまうことなく、マイクロサービスを最大限に活用できるだろうか？　このトピックは章の残りで説明する。

「マイクロサービス」という用語の歴史

2011年、私がまだThoughtWorksでコンサルタントをしていた時のことだ。友人であり、当時の同僚でもあったJames Lewisは、彼が「マイクロアプリ（micro-apps）」と呼んでいたものにとても興味を持っていた。彼は、サービス指向アーキテクチャを使用しているいくつかの企業が、サービスの置き換えを容易にするためにこのパターンを用いていることに気付いたのだ。問題の企業は、特定の機能を迅速に導入することに興味を持ちつつも、全てをスケールアップする必要がある場合には、サービスを他の技術スタックで置き換えられるようにしていた。

当時、目立っていたのは、そうしたサービスのスコープの小ささだった。中には数日で書ける（あるいは書き直せる）ものもあった。Jamesは、「サービスは頭に納まるくらいの大きさにするべきだ」と指摘した。機能のスコープはわかりやすくすべきで、その結果として、変更が容易となると考えたのだ。

その後、2012年になると、Jamesはアーキテクチャについてのサミットでこれらの考えを共有した。そのサミットには、私たち同僚も数名参加していた。そのセッションでは、実際にはこれらのサービスは自己完結型のアプリケーションではないので、「マイクロアプリケーション」は正しくないという事実について議論した。そして、その代わりに、「マイクロサービス（microservices）」という名前の方がより適切だという結論となった[†6]。

[†6] 最初にこの用語を実際に書き記した時のことは思い出せない。しかし、文法上のあらゆる論理に反して、この用語をハイフンでつなぐべきではないと主張したことを鮮明に覚えている。今にして思えば、それは正当化するのが難しい主張だったのだが、それでも私はそれに固執した。そして、理不尽ではあったものの、最終的には私の意見が通ることとなった。

1.1.9　所有権

ビジネスドメインに基づいてモデル化されたマイクロサービスでは、IT成果物（独立してデプロイ可能なサービス群）とビジネスドメインとの間に整合性が見られる。この考え方は、テクノロジー企業が「ビジネス」と「IT」の垣根を取り払う方向にシフトしていることを考えると、うまく響いてくる。図1-4に示すように、従来のIT組織では、要件を定義し顧客と接点を持つビジネス部門とは全く異なる部門がソフトウェア開発を行っていることが多い。この種の組織の機能不全は多種多様であり、おそらくここで詳しく説明する必要はないだろう。

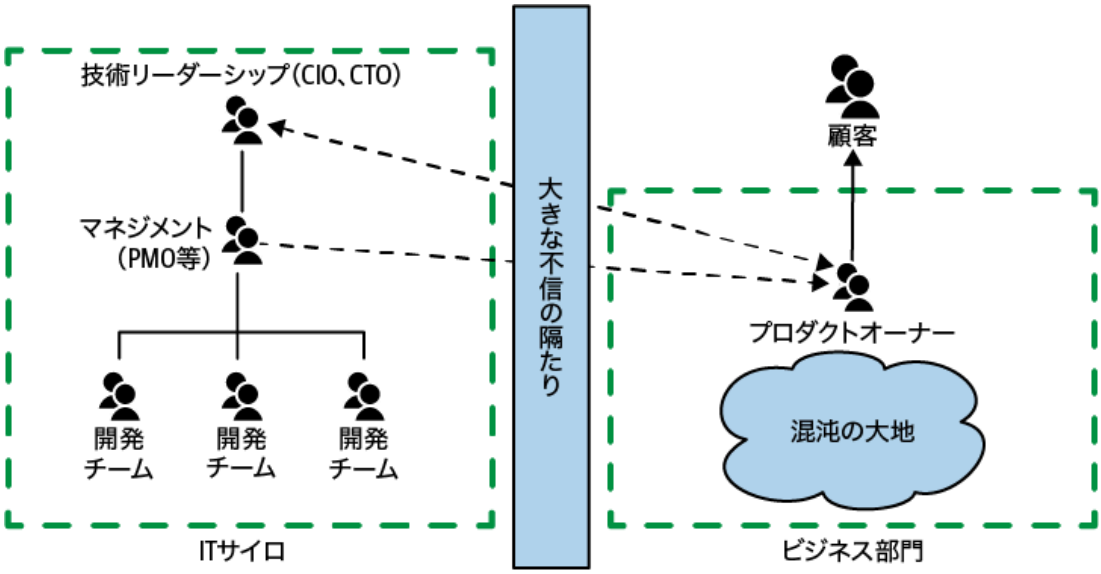

図1-4　従来のIT部門とビジネス部門の組織ビュー

代わりに、真のテクノロジー企業に見られるのは、そうしたばらばらな組織的サイロを完全に結合させている姿だ。これを図1-5に示す。プロダクトオーナーはデリバリーチームの一員として直接仕事を行い、デリバリーチームは任意の技術グループではなく、顧客に向けた製品ラインに基づいて編成されている。中央集権的なIT機能は、組織の標準であるというよりも、このような顧客中心のデリバリーチームをサポートするために存在している。

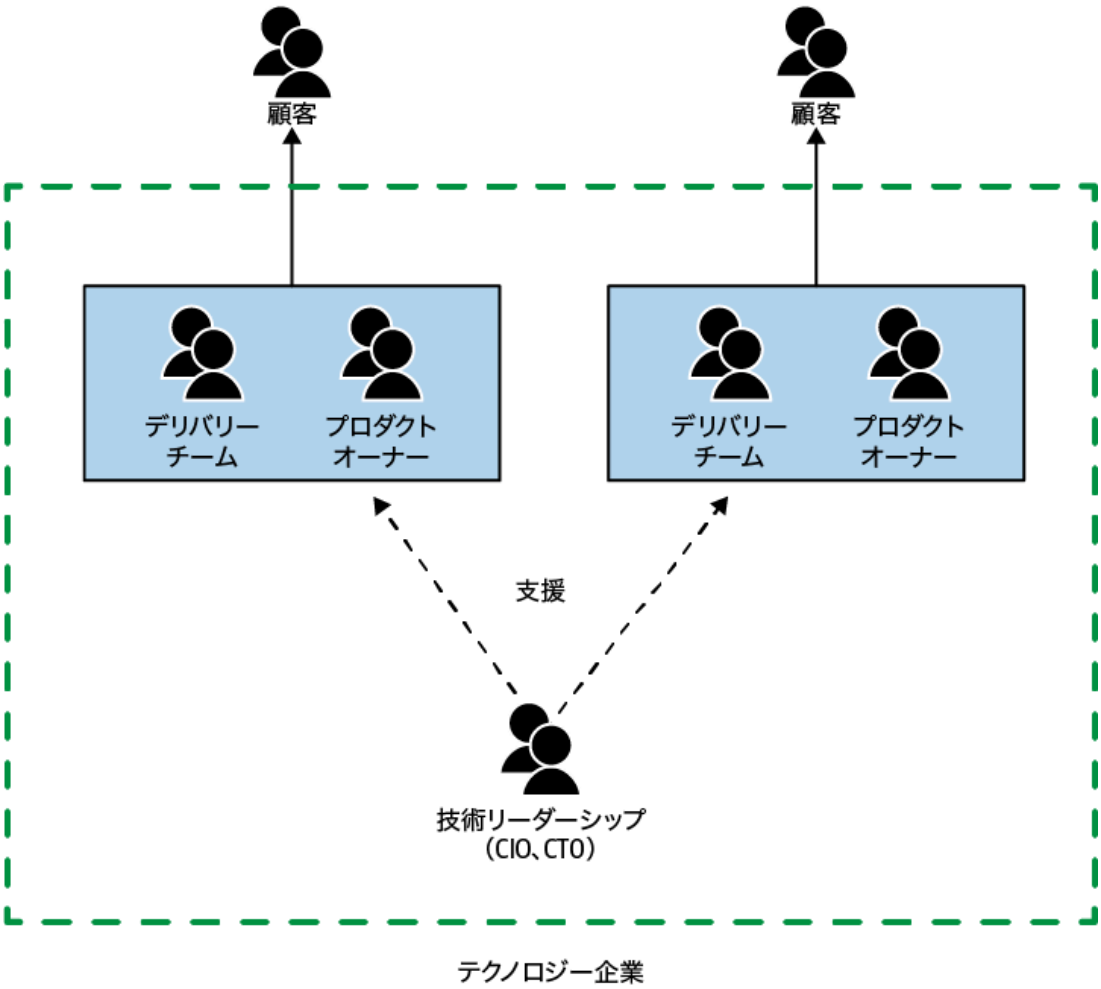

図1-5　真のテクノロジー企業はどのようにソフトウェアデリバリーを統合しているか

全ての組織がこうした形態にシフトしていっているわけではないものの、マイクロサービスアーキテクチャは、こうしたシフトをはるかに容易にする。製品ラインに沿ってデリバリーチームを編成する際、サービスがビジネスドメインに沿って分割されていれば、製品指向のデリバリーチームにサービスを確実に所有させることが容易になる。複数のチームで共有されているサービスを減らすことは、デリバリー衝突を最小限に抑えるための鍵となる。ビジネスドメイン指向のマイクロサービスアーキテクチャは、こうした組織構造のシフトをはるかに容易にする。

1.2　モノリス

ここまでマイクロサービスについて話してきた。しかし、本書はモノリス**から**マイクロサービス**へ**の移行をテーマにしている。そのため、**モノリス**という言葉の意味をはっきりさせておく必要もある。

本書でモノリスについて話すときは、主にデプロイの単位を指している。システム内の全ての機能を一緒にデプロイする必要があったとき、本書ではそれをモノリスと考える。モノリシックなシステムには、単一プロセスのモノリス、分散モノリス、サードパーティ製ブラックボックスシステムの3つの種類がある。

1.2.1　単一プロセスのモノリス

モノリスを議論するときに思い浮かぶ最も一般的な例は、図1-6のように、全てのコードが**単一のプロセス**としてデプロイされているシステムだ。堅牢性やスケーリングのために、システムにこのプロセスのインスタンスを複数持たせることもあるかもしれないが、基本的には全てのコードが単一のプロセスに詰め込まれている。実際には、これらの単一プロセスのシステムは、ほとんど常にデータベースからデータを読み込んだり、データベースにデータを保存したりするため、それ自体が単純な分散システムになり得る。

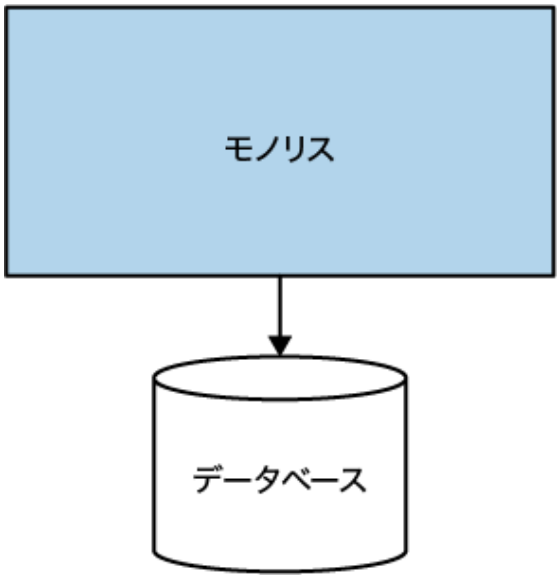

図1-6　単一プロセスのモノリス：全てのコードが単一のプロセスにパッケージ化されている

おそらく、人々が苦労しているモノリシックシステムの圧倒的多数が、こうした単一プロセスのモノリスである。したがって、本書でもこの種類のモノリスについて大部分の時間を割くことになる。今後、本書で「モノリス」という言葉を使うときは、特に断りのない限り、この種類のモノリスを指すことにする。

モジュラーモノリス

単一プロセスのモノリスのサブセットとして、**モジュラーモノリス**がある。モジュラーモノリスとは、単一プロセスが別々のモジュールで構成され、それぞれ独立して作業できるものの、デプロイのために結合する必要があるシステムだ。図1-7にこれを示す。ソフトウェアをモジュールに分解するという概念は新しいものではない。章の後半では、これにまつわる歴史の一部に触れる。

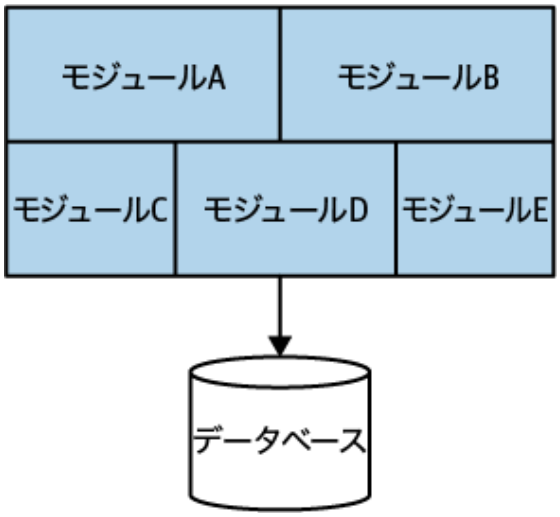

図1-7　モジュラーモノリス：プロセス内のコードが複数のモジュールに分解されている

多くの組織にとって、モジュラーモノリスは優れた選択肢となる。モジュールの境界が明確に定義されていれば、高度な並列作業が可能になる。その上、より分散されたマイクロサービスアーキテクチャの課題も回避でき、デプロイもよりシンプルになる。Shopifyは、マイクロサービス分解の代替としてこの技術を使用している好例であり、実際とてもうまく機能しているらしい[†7]。

[†7] マイクロサービスではなくモジュラーモノリスを使用するShopifyの考え方については、YouTubeにあるKirsten Westeindeの講演（http://bit.ly/2oauZ29）が役に立つ。

モジュラーモノリスの課題の1つは、コードレベルで見られる分解がデータベースで失われてしまう傾向があることだ。これは、将来的にモノリスを分解したくなった場合に重大な課題を引き起こす可能性がある。それに対し、モジュラーモノリスのアイデアをさらに推し進めて、図1-8に示すように、モジュールと同じラインに沿ってデータベースを分解しているチームを見てきたこともあ

る。既存のモノリスにこのような変更を加えることは、たとえコードをそのままにしておくとしても、基本的には非常に困難な作業となる。もし、似たようなことをしようとする際には、4章で解説するさまざまなパターンが役に立つだろう。

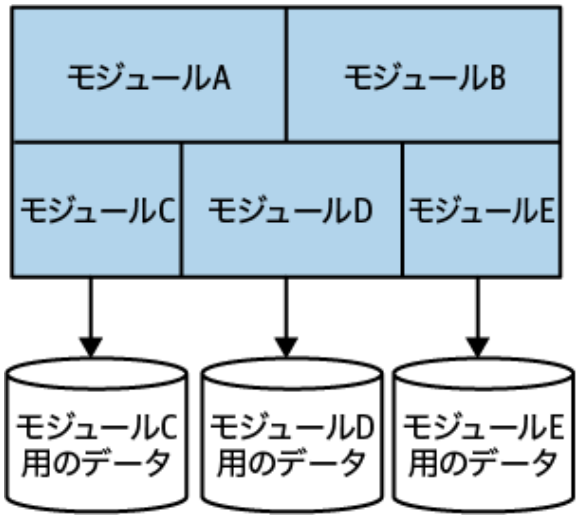

図1-8　分解されたデータベースを持つモジュラーモノリス

1.2.2　分散モノリス

> *分散システムとは、存在すら知らなかったコンピューターの障害で、あなたのコンピューターが使えなくなるようなシステムのことだ[†8]。*
>
> *Leslie Lamport*

[†8] 1987年5月28日23分29秒PDTに、DEC SRC掲示板に送信されたメールメッセージ（https://www.microsoft.com/en-us/research/publication/distribution/）。

分散モノリスとは、複数のサービスで構成されているものの、何らかの理由でシステム全体が一緒にデプロイされなければならなくなっているシステムのことだ。分散モノリスはサービス指向アーキテクチャ（SOA）の定義を満たしているかもしれないが、たいていはSOAの約束は果たせない。私の経験からすると、分散モノリスは分散システムの全ての欠点と単一プロセスモノリスの欠点を持ち、そしてどちらの利点も十分には持たない。過去の仕事で分散モノリスに出会ったことは、私自身のマイクロサービスアーキテクチャに対する興味関心に大きな影響を与えている。

分散モノリスは、典型的には、情報隠蔽やビジネス機能の凝集といった概念に十分な焦点が当てられていない環境で発生する。そうした環境では、サービスの境界を越えて変更が波及し、その結果、局所的には一見無害に見える変更がシステムの他の部分を壊すような、密結合となったアーキテクチャが生み出されることになる。

1.2.3　サードパーティ製のブラックボックスシステム

移行の取り組みの中で「分解」したくなった**サードパーティ製のソフトウェア**をモノリスと捉えることもできる。給与システム、CRMシステム、人事システムなどがこれに含まれるかもしれない。共通しているのは、それが他の誰かに開発されたソフトウェアであり、自分たちではコードを変更できないということだ。それは、自社インフラストラクチャにデプロイされた既製ソフトウェアの場合もあれば、自社で使用しているSaaS（Software-as-a-Service）製品の場合もある。本書で紹介する分解技術の多くは、内部のコードを変更できないシステムでも使用できるものになっている。

1.2.4　モノリスの課題

単一プロセスのモノリスであれ分散モノリスであれ、モノリスは結合、特に実装時とデプロイ時の結合の危険に対してより脆弱なことが多い。これについてはすぐ後で探っていく。

同じ場所で作業する人が増えれば増えるほど、お互いの邪魔になる。異なる開発者が同じコードを変更したいと思ったり、異なるチームが異なる時間帯に機能をプッシュしたいと思ったり（あるいはデプロイを遅らせたいと思ったり）する。そこでは、誰が何を所有しているのか、誰が意思決定をするのかをめぐる混乱が生じる。多くの研究が、所有権の境界線が混乱することの課題を示している[†9]。私はこの問題を**デリバリー衝突**と呼んでいる。

[†9] マイクロソフトリサーチはこの分野の研究を行っており、その全てがお勧めだ。出発点としては、Christian Birdらによる “Don't Touch My Code! Examining the Effects of Ownership on Software Quality（私のコードに触れるな！　ソフトウェア品質に対する所有権の影響に関する調査）”[2]をお勧めする。

モノリスだからといってデリバリー衝突の問題に必ず直面するわけではないし、マイクロサービスアーキテクチャだからといってこの問題に直面しないというわけではない。しかし、マイクロサービスアーキテクチャを採用すると、具体的なシステムの境界を与えてくれるため、所有権の線を引けるようになり、この問題をどのように軽減するかについて、より高い柔軟性が得られる。

1.2.5　モノリスの利点

しかしながら、単一プロセスのモノリスにも多くの利点がある。そのはるかにシンプルなデプロイ方式によって、分散システムにおける多くの落とし穴を回避できる。また、監視、トラブルシューティング、エンドツーエンドテストなどの作業もずっと簡単になる。

モノリスの場合は、その中でコードを再利用するのも簡単だ。分散システムでコードを再利用したい場合には、コードをコピーするか、ライブラリに切り出すか、共有機能を独立したサービスにするかを決めなければならない。モノリスの場合は、コードを再利用する方法はずっとシンプルだ。全部のコードがそこにあるのだから、それを使えばいいのだ。そして、多くの人がそのシンプルさを気に入っている。

しかし残念なことに、人々はモノリスを避けるべきもの、つまり本質的に問題のあるものとして見るようになった。私は、**モノリス**という用語を**レガシー**と同義と見なす何人もの人々と出会ってきた。これは問題だ。モノリシックアーキテクチャは選択肢の一つであり、それは今でも有効な選択だ。マイクロサービスと同様に、全ての状況で正しい選択ではないかもしれないだけだ。もしソフトウェアを提供するための実行可能な選択肢としてのモノリスを、手当たり次第に否定するという罠に陥ってしまったら、私たちは自分自身やソフトウェアのユーザーに対して適切でない行動を取るリスクを負うことになる。モノリスとマイクロサービスのトレードオフについては、3章でさらに掘り下げ、それぞれ固有のコンテキストごとの正解を評価するのに役立つツールについて説明していく。

1.3　結合と凝集

マイクロサービスの境界を定義する際には、結合と凝集のバランスを理解することが重要だ。**結合（Coupling）**とはあるものに対する変更がどれくらい別のものの変更を必要とするかを表し、**凝集（Cohesion）**とは関連するコード

をどのようにグループ化するかを表す。これらの概念は直接つながりがある。次に示すコンスタンティンの法則は、この関係をうまく表現している。

> *構造は、凝集度が高く、結合度が低い場合に安定する。*
>
> *Larry Constantine*

これは思慮深く有用な考察だ。密接に関連する2つのコードがあるとき、関連する機能は両方にまたがることになるため、凝集度は低くなる。そして、片方のコードが変更されると、もう片方のコードも変更が必要になるため、結合度も高くなる。

分散システムでは、システムの構造を変更する際のコストは高くつく。サービス境界を越えた変更のコストはとても高くなるからだ。1つ以上の独立してデプロイ可能なサービスに変更を加えなければならず、おそらくサービス契約に対する破壊的変更に対処する必要がある。これは大きな足かせになるだろう。

モノリスの問題は、多くの場合がその逆になってしまうことだ。凝集に向かい、一緒に変更しがちなものだけをまとめておくのではなく、関係のないあらゆる種類のコードを取得して一緒にくっつけてしまう。同様に、疎結合は実際には存在しない。あるコード行に変更を加えるのは容易かもしれないが、その変更をモノリスの残りの部分に影響を与えることなくデプロイすることはできず、システム全体を必ず再デプロイしなければならない。

独立デプロイ可能性の考え方をできる限り満たすには、システムを安定させる必要もある。自分たちのサービスに加えた変更を、**他のサービスに変更を加えることなしに**本番環境にデプロイできるようにするには、使用しているサービスが安定している必要があり、サービスの利用者に安定した契約を提供しなくてはならないからだ。

結合や凝集についての情報は既に豊富にあるので、ここで改めて多くを語るまでもないだろう。しかし、特にマイクロサービスアーキテクチャの文脈にこれらの考えを取り入れるには、要旨をまとめておくべきだと考えている。つまるところ、凝集と結合の考えは、マイクロサービスアーキテクチャをどのように考えるかに大きな影響を与える。そして、これは驚くべきことではない。なぜなら、凝集と結合はモジュール化されたソフトウェアに関する考え方であり、マイクロサービスアーキテクチャとは、ネットワークを介して通信し、独立してデプロイできるモジュール以外の何者でもないからだ。

結合と凝集の小史

結合と凝集は、1968年にLarry Constantineによって最初に発案された考えで、長い間この業界に存在してきた。この結合と凝集という双子の考えは、コンピュータープログラムを書くことについて考える際の多くの基礎を形づくっている。Larry ConstantineとEdward Yourdonによる『ソフトウェアの構造化設計法』（日本コンピュータ協会）[36]のような書籍は、その後、何世代にもわたるプログラマーに影響を与えた（私自身も、この本は大学の学位取得のための必読書だった。この本が出版されてから20年近く過ぎているにもかかわらずだ）。

Larryが最初に凝集と結合の考え方を説明したのは1968年、モジュール式プログラミングに関する全米シンポジウムにおいてだった。コンウェイの法則が生まれたのも同カンファレンスで、1968年はコンピューター業界にとって特に恵まれた年だった。この年には、今では悪名高いNATOが主催となった2つの会議も開催され、同会議では、考え方としてのソフトウェアエンジニアリングも注目を集めた（用語自体はそれ以前にMargaret H. Hamiltonによって造られていた）。

1.3.1　凝集

これまでに聞いた中で最も簡潔な凝集の定義は、「共にあり、共に変更されるコード」というものだ。私たちの目的にとって、この定義はかなり良い。既に述べたように、私たちはビジネス機能の変更しやすさに基づいてマイクロサービスアーキテクチャを最適化する。そのため、できるだけ変更箇所が少なくなるような方法で機能をグループ化したいと考えている。

請求書の承認方法を変更したいときに、変更が必要な箇所を複数のサービスをまたいで探し出したりはしたくないし、機能を公開するために変更を施したサービス間でリリースを調整したくもないからだ。その代わりに、変更コストをできる限り低く抑えられるよう、変更を加えるサービスをできるだけ少なくしたいと考えている。

1.3.2　結合

情報隠蔽は、ダイエットと同じで、実際に行うよりも若干簡単に説明される。

David Parnas “The Secret History Of Information Hiding” [28]

凝集が好ましい一方で、結合には注意が必要だ。多くのものが「結合」すればするほど、一緒に変更しなければならないものが多くなる。ただし、結合には種類があり、種類に応じて対応策が異なる可能性がある。

結合の分類にあたっては、Meyer、Yourdan、Constantineによる仕事をはじめ、多くの先行研究がある。これから結合の分類を示すにあたって、それに文句をつけるつもりはないことをわかってほしい。ここで示すのは、分散システムにおける結合に関する側面の理解にとても役立つ分類だ。したがって、結合のさまざまな形態を網羅的に分類するつもりはない。

情報隠蔽

結合に関する議論になると幾度となく出てくる概念に **情報隠蔽（Information Hiding）**がある。この概念は、1971年にDavid Parnasが最初に提唱したもので、モジュールの境界をどのように定義するかという彼の研究から生まれた[†10]。

情報隠蔽の本質的な考え方は、頻繁に変更されるコードを静的な部分から分離することにある。モジュールの境界を安定したものにし、モジュールの実装で頻繁に変更されるであろう部分を隠す。モジュールの互換性が維持されている限り、内部の変更を安全に行えるという考え方だ。

個人的には、モジュール (またはマイクロサービス) の境界で公開するものは可能な限り少なくするアプローチを採用している。いったんモジュールインターフェイスの一部になってしまったものを、さかのぼって隠すのは困難だからだ。逆に、隠蔽しておいたものを後から共有するのはいつでも行える。

オブジェクト指向プログラミングの概念としてのカプセル化は、情報隠蔽に関連している。しかし、どの定義を受け入れるかによっては、同じ意味ではない可能性がある。オブジェクト指向プログラミングにおけるカプセル化とは、1つ以上のものを1つのコンテナにまとめることを指すようになった。フィールドとそれらを扱うメソッドを含むクラスを考えると良いだろう。オブジェクト指向プログラミングでは、クラスを定義する際に可視性を使用することで実装の一部を隠蔽できるようになっている。

情報隠蔽の歴史をもっと詳しく知りたい場合は、Parnasの “The Secret History of Information Hiding”（Springer）[28]をお勧めする。

[†10] 情報隠蔽の起源として引用されることが多い論文に、1972年に出されたParnasの有名な論文 “On the Criteria to be Used in Decomposing Systems into Modules” があるが、彼が最初にこの考え方を発表したのは“Information Distributions Aspects of Design Methodology”（Proceedings of IFIP Congress '71）[27]においてだった。

実装結合

実装結合は、私が目にする結合の中で最も悪質なものだが、幸いなことに、最も低減しやすいものの一つだ。実装結合は、Bの実装方法によってAがBに結合されるという結合だ。この場合、Bの実装を変更するときには、Aも変更する必要がある。

ここでの問題は、実装の詳細はしばしば開発者の好みで選ばれることだ。ある問題を解く方法がたくさんあるとき、私たちはどれかを選ぶが、いずれ方針を変えることもあるだろう。私たちが方針を変えると決めたときでも、それによってコンシューマー[†11]を壊したくはない（独立デプロイ可能性のこと、覚えているだろうか？）。

[†11] 訳注：サービスの利用者のこと。

実装結合の古典的で一般的な例として、データベースを共有するという形がある。図1-9では、注文サービスはシステムで発注された全注文のレコードを含んでいる。レコメンデーションサービスは、過去の購入履歴に基づき顧客が購入したいだろうレコードをレコメンドする。現在、レコメンデーションサービスは、データベース上でこのデータに直接アクセスしている。

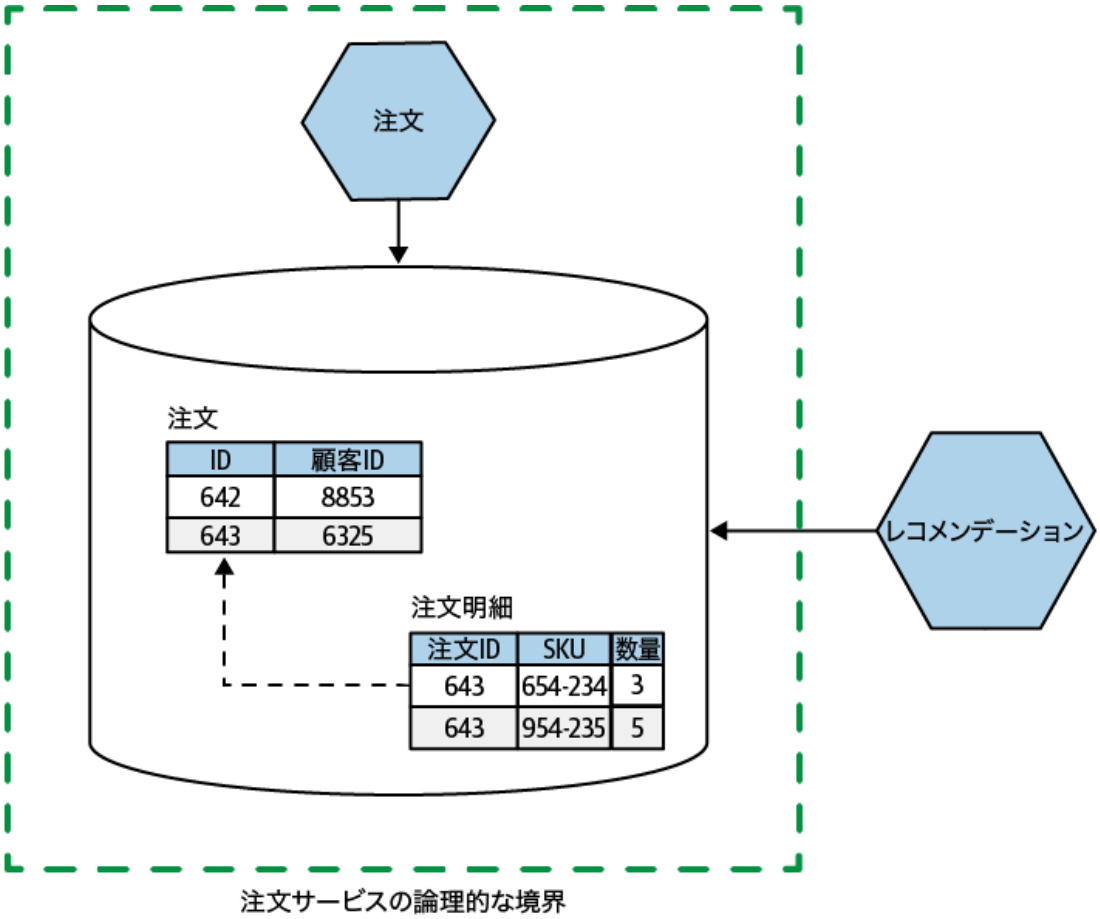

図1-9　レコメンデーションサービスは注文サービス内に格納されているデータに直接アクセスする

レコメンデーションには、どの注文が行われたかの情報が必要だ。これは、ある程度は避けられない**ドメイン結合**だ。しかし、この状況では、特定のスキーマ構造、SQLの方言、そして行の内容にさえも結合されている。注文サービスが列の名前を変更したり、顧客注文テーブルを分割したりした場合、概念としての注文情報はまだ存在するものの、レコメンデーションサービスはそこから注文情報を取得できなくなってしまう。図1-10に示すように、お勧めなのはこの実装詳細を隠蔽することだ。この図では、レコメンデーションサービスはAPI呼び出しを介して必要な情報にアクセスしている。

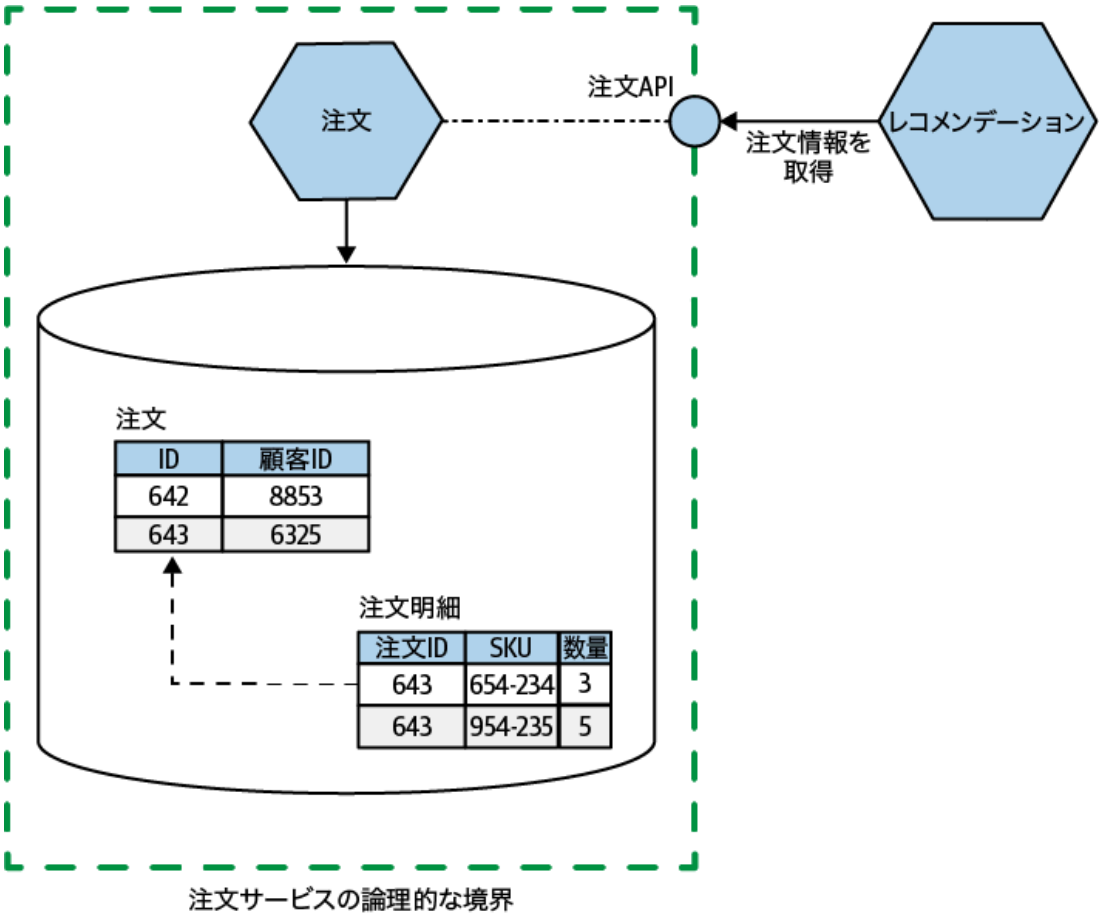

図1-10　レコメンデーションサービスはAPI経由で注文情報にアクセスし、内部実装の詳細を隠す

また、図1-11のように、注文サービスにデータベースの形でデータセットを公開して、コンシューマーが一括アクセスできるようにする方法もある。注文サービスがそれに応じてデータを公開できる限り、注文サービスの内部で行われた変更はコンシューマーからは見えない。注文サービスがパブリックな契約を維持するからだ。これによって、コンシューマーに向けて公開したデータモデルを、ニーズに合わせて改善する機会も開ける。このようなパターンについては、3章や4章で詳しく解説する。

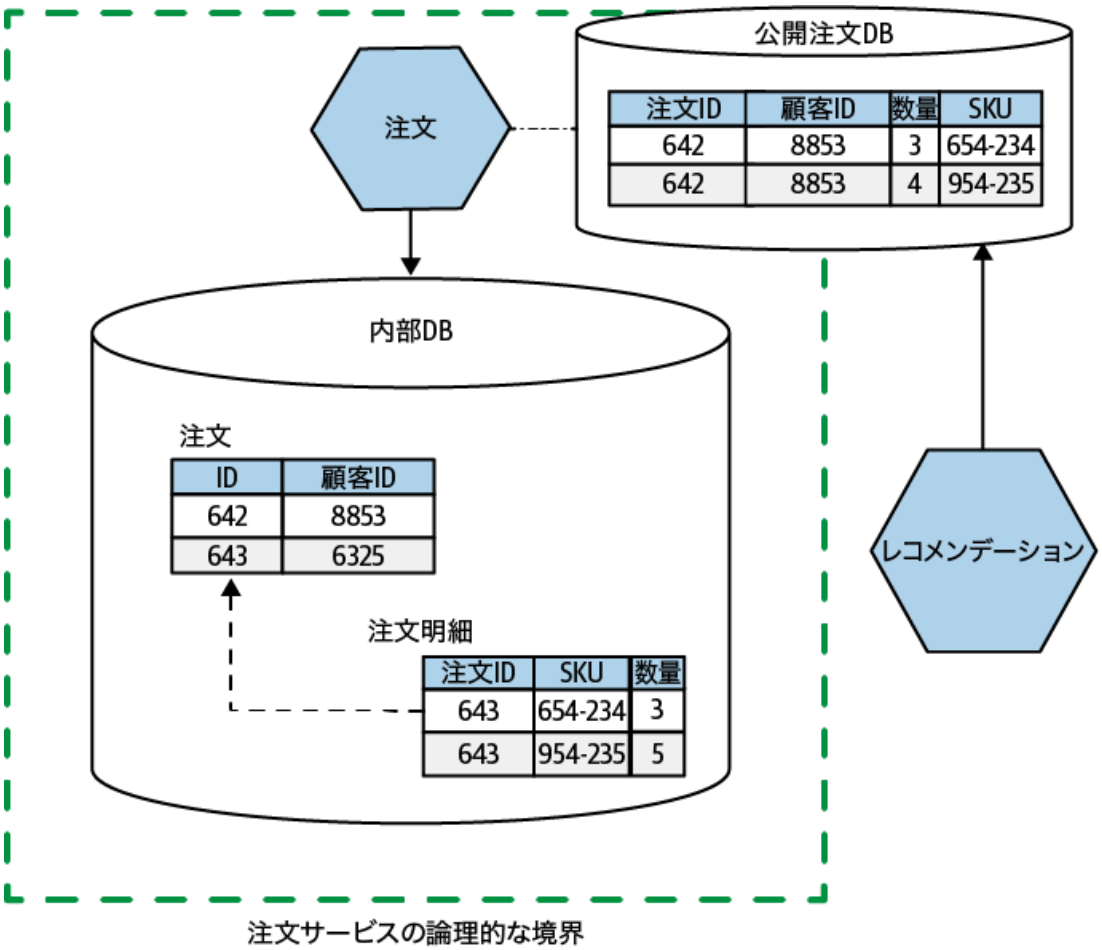

図1-11　レコメンデーションサービスは公開データベース経由で注文情報にアクセスする。公開データベースは内部データベースとは異なる構造を持つ

前述の例は、どちらも実際に情報隠蔽を利用している。明確なサービスインターフェイスの後ろにデータベースを隠すという行為によって、サービスが公開される範囲の制限と、データ表現の仕方の変更が可能になる。

サービスインターフェイスを定義する際に「アウトサイドイン」思考[†12]を使うのも効果的な方法だ。まず、サービスコンシューマーの視点から物事を考えることでサービスのインターフェイスを導き出す。次に、そのサービス契約の実現方法を考え出す。しかし、私が代わりによく目にするアプローチはその逆だ（残念ながらこちらの方があまりにも一般的だ）。サービスに取り組むチームは、データモデルや別の内部実装の詳細を理解してから、それを外部に公開することを考える。

[†12] 訳注：「サービスがどのように利用されるか」という、サービス利用者の観点からサービスをデザインしていく考え方。

「アウトサイドイン」思考では、まず「サービスのコンシューマーは何を必要としているのか？」を問う。これは、コンシューマーが何を必要としているかを**自問する**という意味ではない。サービスを呼び出す人々に実際に尋ねるという意味だ。

マイクロサービスが公開するサービスインターフェイスをユーザーインターフェイスのように扱おう。アウトサイドインの思考を使って、サービスを呼び出す人々と共同でインターフェイスのデザインを形成しよう。

外界とのサービス契約をユーザーインターフェイスだと考えてほしい。ユーザーインターフェイスを設計する際には、ユーザーが何を求めているのかをユーザーに尋ね、それをユーザーと一緒に設計していくことを繰り返し行う。サービス契約も同じように形にすべきだ。その方がコンシューマーにとってより使いやすいサービスになるという事実をひとまずおいても、そうすることは、外部契約と内部実装をある程度切り離すのにも役立つ。

一時的結合

一時的結合は、主に実行時の懸念事項であり、分散環境での同期呼び出しが持つ主要な課題だ。メッセージ送信とそのメッセージを処理する方法が時間的につながっている状態を、一時的結合があると言う。この表現はちょっとわかりにくいので、図1-12に明示的な例を示そう。

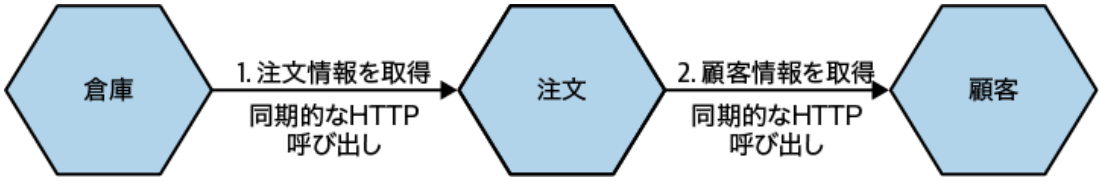

図1-12　同期呼び出しを使用して処理を行う3つのサービスは一時的に結合していると言える

図1-12では、注文に関する必要な情報を取得するため、倉庫サービスから注文サービスに対して同期的なHTTP呼び出しが行われている。そして、注文サービスは顧客サービスから情報を取得するために、再び同期的なHTTP呼び出しを行っている。この全体的な処理を完了するには、倉庫サービス、注文サービス、顧客サービスを全て起動し、連携できるようにする必要がある。すなわち、これらのサービスは一時的に結合されていると言える。

この問題を減らす方法はいろいろある。まず、キャッシュの使用を検討できる。もし顧客サービスから取得している情報をキャッシュしたなら、場合によって注文サービスは顧客サービスとの一時的結合を回避できる。次に、リクエストの送信に非同期トランスポートを利用することも検討できる。その場合には、おそらくメッセージブローカーのようなものを使用することになる。非同期トランスポートを利用すれば、メッセージを下流サービスに送信し、下流サービスが利用可能になった段階でそのメッセージを処理できる。

サービス間通信の種類を網羅することは本書の範囲外だが、マイクロサービスに関する部分については4章で詳しく説明する。

デプロイ時結合

静的にリンクされた複数のモジュールからなる単一のプロセスを考えてみよう。あるモジュールで1行のコードに変更が加えられ、その変更をデプロイしたいとする。そのためには、変更されていないモジュールも含めて、モノリス全体をデプロイする必要がある。このように、全てを一緒にデプロイしなければならないものを、**デプロイ時結合**があると言う。

デプロイ時結合は、静的にリンクされたプロセスの例のように強制されることもあるが、リリーストレイン†13のようなプラクティスが強いる選択の問題でもある。リリーストレインでは、リリーススケジュールを事前に計画するのだが、そのスケジュールは大体において繰り返すものとして計画される。リリースの期限が来ると、最後のリリーストレイン以降に行われた全ての変更がデプロイされる。一部の人にとっては、リリーストレインは有用なテクニックだ。しかし、私はそれを最終的な到達点ではなく、適切なリリースオンデマンド手法に向かうための途中のステップとして扱うことを強く望んでいる。私はかつて、リリーストレインのプロセスの一部として、システム内の全てのサービスを一度にデプロイしてしまう組織で働いたことがあった。その組織の人々は、それらのサービスが変更される必要があるかどうかさえ考えていなかった。

[†13] 訳注：リリーストレインという名前は、この説明にあるように、リリースを時間通りに刻々と発車する電車（トレイン）のように考えることに由来する。

何かをデプロイすることはリスクを伴う。デプロイのリスクを減らす方法はたくさんあるが、その方法の1つに、変更する必要のあるものだけをデプロイするというものがある。より大きなプロセスを、独立してデプロイ可能なマイクロサービスに分解するなどしてデプロイ時結合を減らせられれば、デプロイの範囲が狭まり、それによってデプロイごとのリスクを小さくできるだろう。

リリースの規模が小さいほど、リスクも小さくなる。問題が発生することが少なくなるからだ。何か問題が発生しても、変更範囲が少ないため、何が問題だったのか、どのように修正すればいいのかを考えるのが簡単になる。リリースの規模を小さくする方法を見つけることは、迅速なフィードバックとリリースオンデマンド方式の重要性を説く継続的デリバリー[†14]の核心に迫るものだ。リリースのスコープが小さければ小さいほど、簡単で安全に公開でき、フィードバックも早く得られる。私自身がマイクロサービスに興味を持ったのは、以前に継続的デリバリーに注目していたからだ。当時、私は継続的デリバリーの採用を容易にするアーキテクチャを探していて、マイクロサービスに興味を持った。

[†14] Jez Humble、David Farley『継続的デリバリー』（KADOKAWA/アスキードワンゴ）[17]参照。

もちろん、デプロイ時結合を減らすためにマイクロサービスが必須となるわけではない。Erlangのようなランタイムは、実行中のプロセスに新しいバージョンのモジュールをホットデプロイすることを可能にするし、最終的には、私たちが日常的に使っている技術スタックの中で、このような機能を利用できることが増えるかもしれない[†15]。

[†15] Greenspunの第10規則は「全ての十分に複雑なCもしくはFortranプログラムは、後付けの、不完全な仕様とバグを持ち、遅い、Common Lispの半

分の実装を含んでいる」というものだ。このジョークは次の新しいジョークに上書きされた。「全てのマイクロサービスアーキテクチャは半分壊れたErlangの再実装が含まれている」 これには多くの真実が含まれていると私は考えている。

ドメイン結合

基本的に、複数の独立したサービスからなるシステムでは、参加サービス間に何らかの相互作用が存在する。マイクロサービスアーキテクチャでは、**ドメイン結合**がその答えとなる。サービス間の相互作用は、実ドメインにおける相互作用をモデル化したものとなる。注文をしたい場合には、顧客の買い物かごにどんな商品が入っているかを知る必要がある。製品を出荷したい場合には、どこに出荷するかを知る必要がある。マイクロサービスアーキテクチャでは、定義上、こうした情報は異なるサービスに含まれている可能性がある。

具体例として、Music Corpを考えてみよう。Music Corpには商品を保管する倉庫がある。顧客がCDを注文した際、倉庫で働く人々はどの商品を梱包してどこに発送するかを知る必要がある。そのため、注文に関する情報は、倉庫で働く人々に共有されなくてはならない。

図1-13は、この例を示している。注文処理サービスは、注文の詳細全てを倉庫サービスに送信する。この操作の一部として、倉庫サービスは顧客IDを使用して顧客サービスから顧客に関する情報を取得し、注文が発送されたときにそれを顧客に通知する方法を把握する。

ここでは注文情報全てを倉庫と共有しているが、これは妥当ではない可能性がある。倉庫が必要とするのは、何を梱包してどこに送るかという情報だけだからだ。倉庫は、商品の価格を知る必要はない（パッケージに請求書を添付する必要がある場合は、事前にレンダリングされたPDFとして渡すことが可能だ）。また、アクセスを制限しなければならない情報があまりにも広く共有されてしまうことにも問題がある。注文全体を共有していると、たとえば、必要のないサービスにクレジットカードの詳細が公開されてしまう可能性があるからだ。

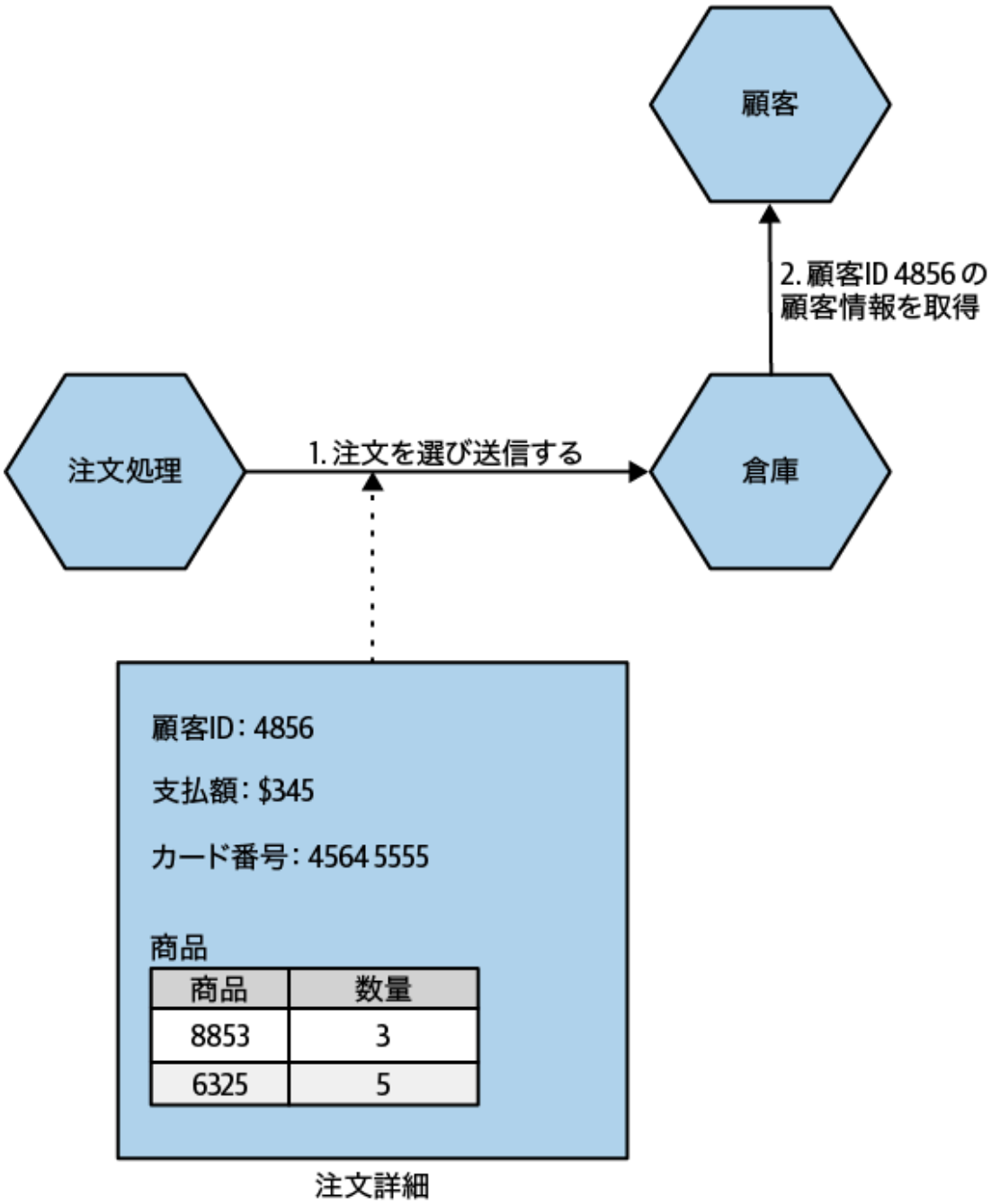

図1-13　注文が倉庫に送られ、配送が開始される

代わりに、図1-14に示すような、倉庫サービスが必要とする情報だけを含む「配送指示」という新しいドメイン概念を考えることもできる。これは情報隠蔽のもう一つの例だ。

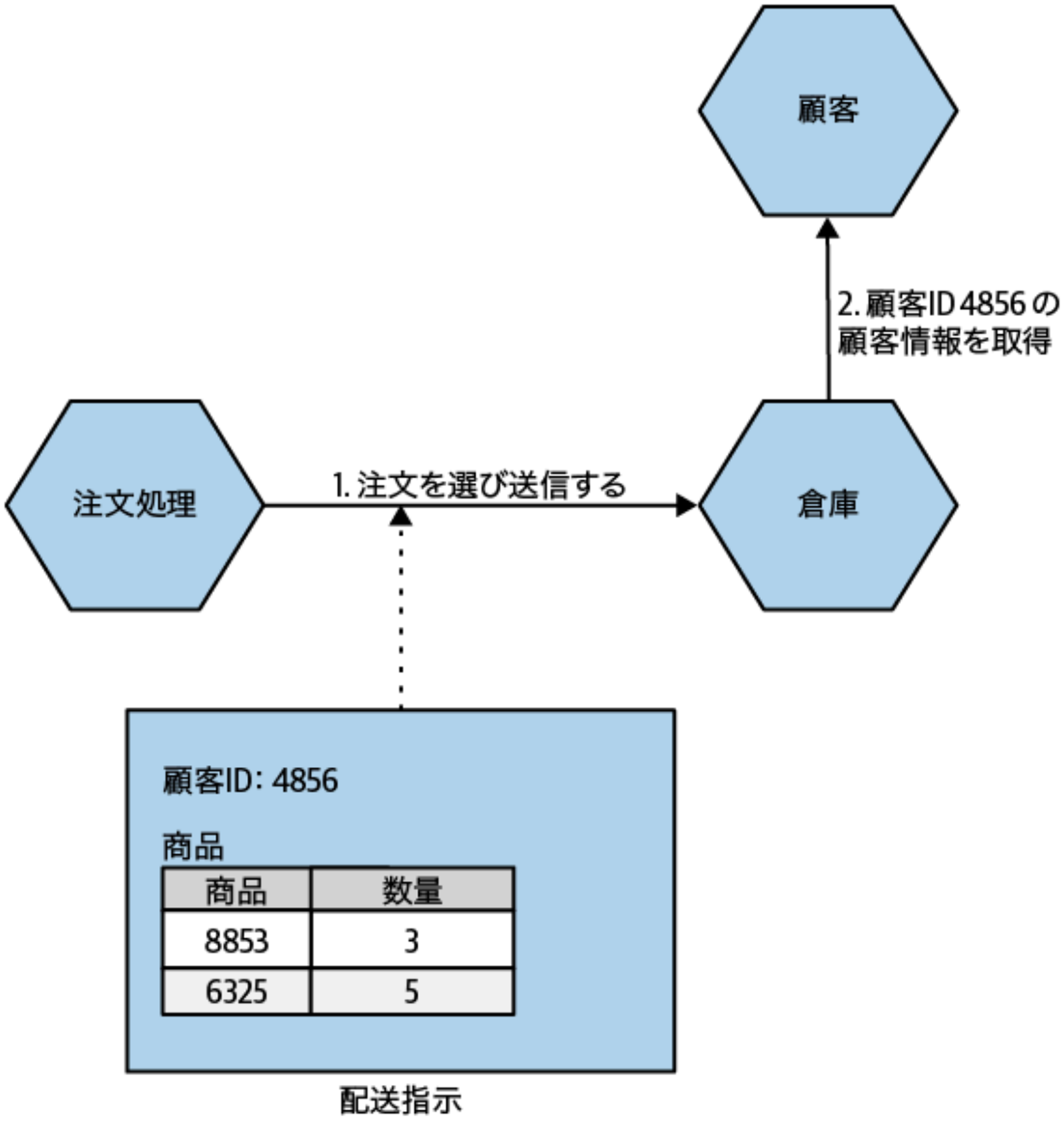

図1-14　配送指示を送ることで、倉庫へ送る情報を減らす

倉庫サービスが顧客のことを知らなくて済むようにすれば、必要に応じて結合をさらに減らせられる。たとえば、図1-15に示すように、配送指示を介して全ての適切な詳細を提供できる。

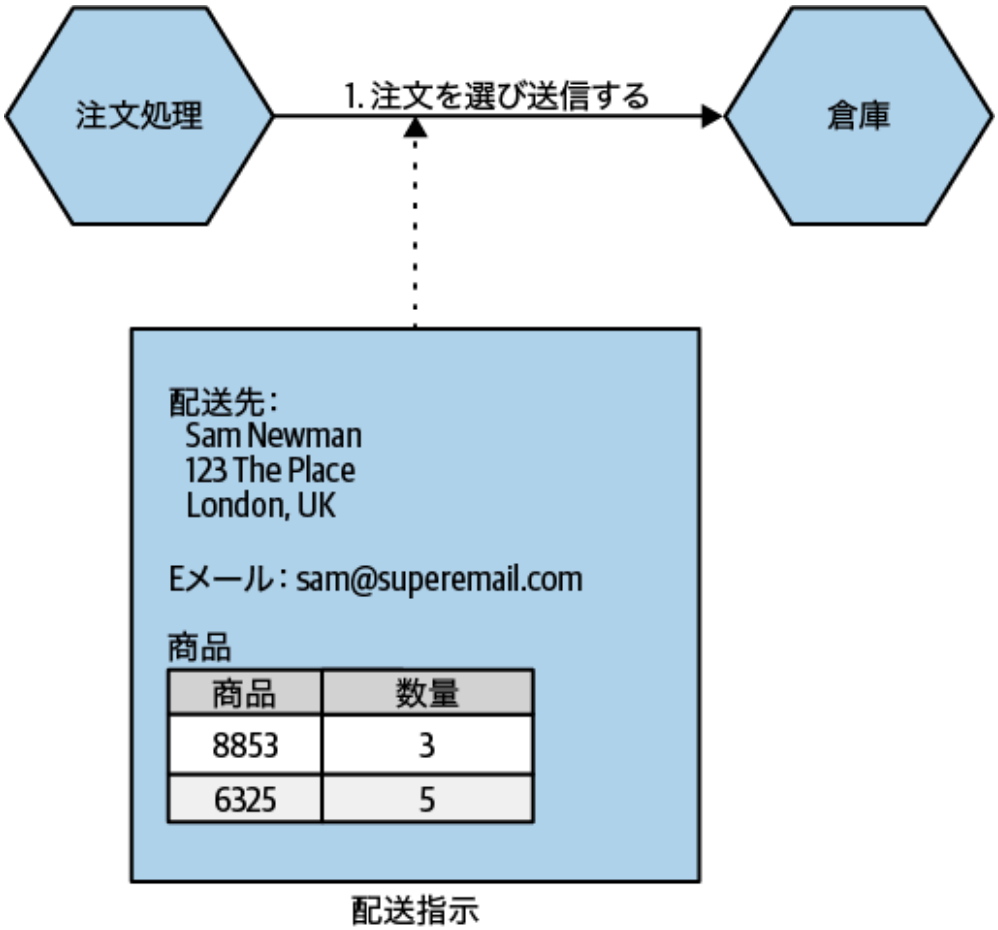

図1-15　配送指示により多くの情報を記載することで、顧客サービスへの問い合わせの必要性を回避できる

このアプローチが機能するには、どこかの時点でこの配送指示を生成できるよう注文処理が顧客サービスにアクセスしなければならないだろう。しかし、注文処理は他の理由でも顧客情報にアクセスする必要があるだろうから、これはどうということはない。ここで、配送指示を「送信」するプロセスとは、注文処理から倉庫サービスへのAPI呼び出しが行われることを表している。

別の方法としては、図1-16のように、倉庫サービスが使用する何らかのイベントを注文処理サービスが発行する方法も考えられる。倉庫サービスが使用するイベントを発行することで、依存関係をうまく反転できる。注文が確実に発送されることを保証できるように注文処理が倉庫サービスに依存する形から、倉庫サービスが注文処理サービスからのイベントを待ち受ける形へと変わるからだ。どちらのアプローチにもメリットがあるが、どちらを選択するかは、注文処理ロジックと倉庫サービスにカプセル化された機能間の相互作用をより広く理解しているかどうかにかかっている。この方法を取る場合には、何らかのドメインモデリングが役立つ。このトピックについては、次で探っていく。

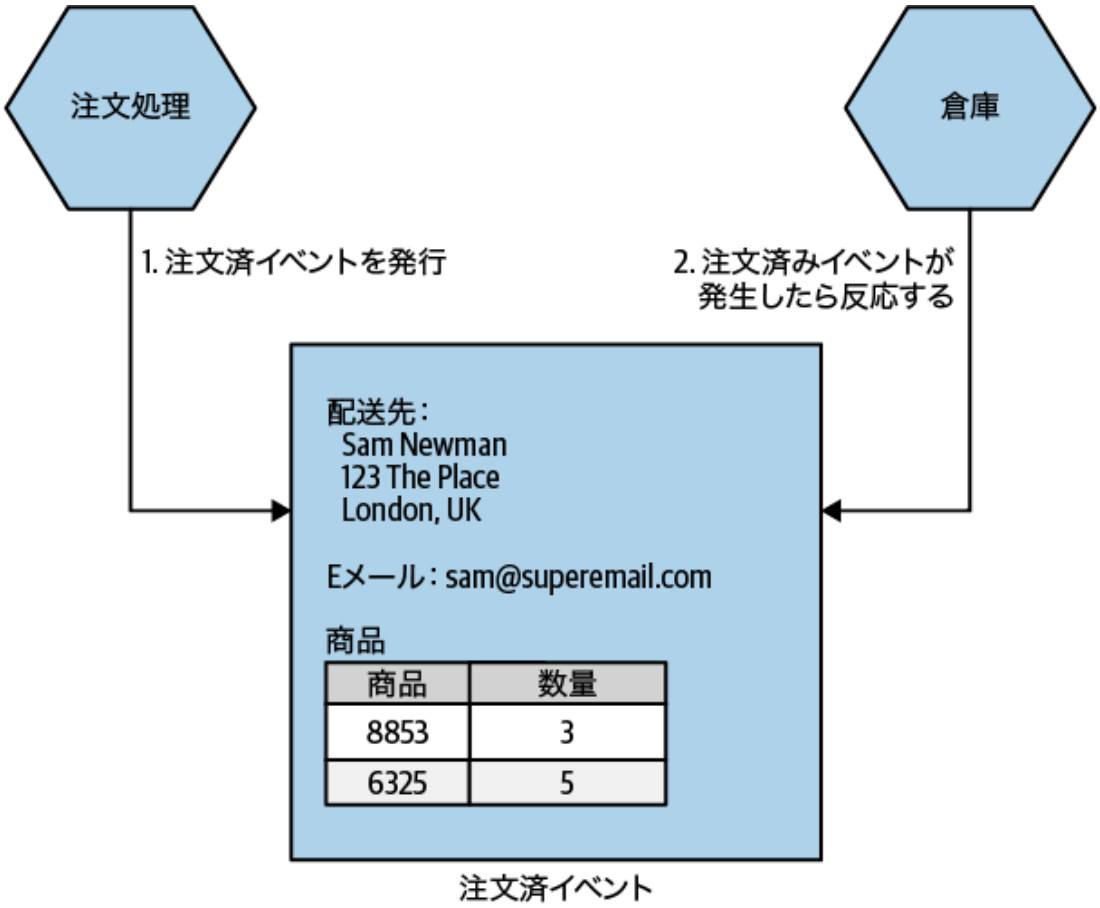

図1-16　倉庫サービスが受け取れるイベントを発行する。イベントには、注文を梱包して発送するのに十分な情報を含む

基本的に、倉庫サービスが何らかの作業を行うには、注文に関する何らかの情報が必要だ。このレベルのドメイン結合は避けることはできない。しかし、ドメイン概念のうち、何をどう共有するかを慎重に考えることによって、生じる結合の度合いは下げられる。

1.4　必要十分なドメイン駆動設計

既に述べたように、ビジネスドメインに基づいてサービスをモデル化することは、マイクロサービスアーキテクチャにとって大きなメリットがある。問題は、そのモデルをどうやって思いつくかだ。そこで、**ドメイン駆動設計（Domain-Driven Design：DDD）**の出番となる。

プログラム自体が動作する実世界をプログラムの中でより良く表現したいという願望は、新しいアイデアではない。Simulaのようなオブジェクト指向プログラミング言語は、実際のドメインをモデル化するために開発された。しかし、このアイデアを実際に形にするには、プログラム言語の能力以上のものが必要だ。

Eric Evansの『エリック・エヴァンスのドメイン駆動設計』（翔泳社）[8]では、問題のドメインをプログラムでよりよく表現するための一連の重要なアイデアが示されている。これらのアイデアを完全に探求することは本書の範囲外だが、マイクロサービスアーキテクチャを考える上で最も重要なアイデアの概要を簡単に説明していこう。

1.4.1　集約

DDDにおける**集約（Aggregate）**は、多くの異なる定義が存在している、やや混乱を招く概念だ。集約とは、ただの任意オブジェクトの集まりなのだろうか？　それともデータベースから取り出すべき最小単位なのだろうか？　私にとって常に役立ってきたモデルは、集約をまず実際のドメイン概念の表現として考えてみるというものだ。注文、請求書、在庫品などを考えてみてほしい。集約は一般的にライフサイクルを持っているので、状態マシンとして実装できる。そして、状態遷移を処理するコードが状態そのものと一緒にグループ化された自己完結型のユニットとして取り扱う。

集約とマイクロサービスの関係でいうと、1つのマイクロサービスが1つ以上の異なる集約のライフサイクルとデータの保存を処理する。他のサービスの機能が、いずれかの集約の状態を変更したい場合には、その集約内での変更を直接要求するか、集約自体がシステム内の他のものに反応して状態遷移を開始させる必要がある。この例を図1-17に示す。

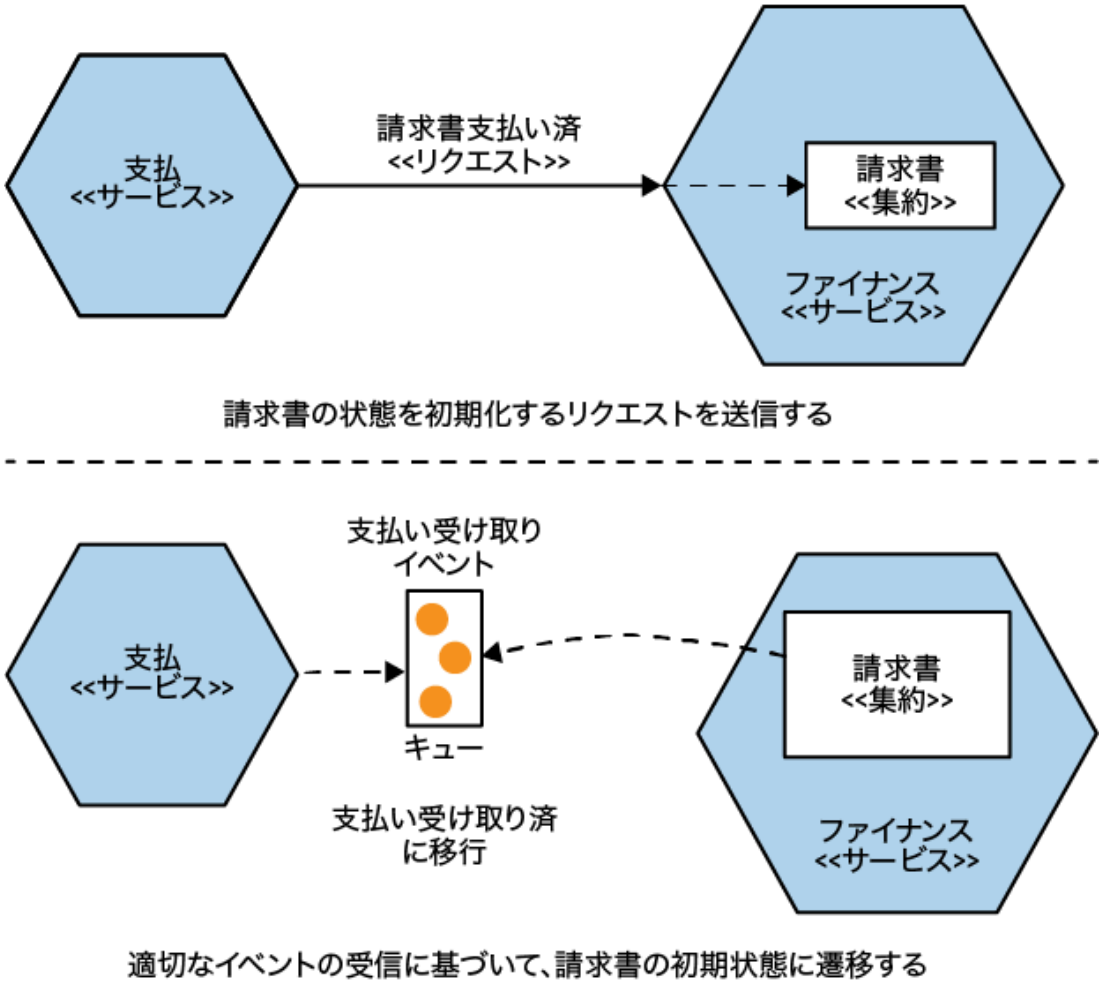

図1-17　支払サービスが請求書集約を支払い済みに遷移させるさまざまな方法

ここで理解すべき重要なことは、状態遷移を外部から要求された場合、集約はノーと言えるということだ。不正な状態遷移が行われないように集約を実装するのが理想だ。

集約は他の集約と関連を持てる。図1-18では、顧客集約は0個以上の注文に関連付けられている。私たちは、異なるサービスによって処理される可能性のある顧客と注文を、それぞれ別の集約としてモデル化することにした。

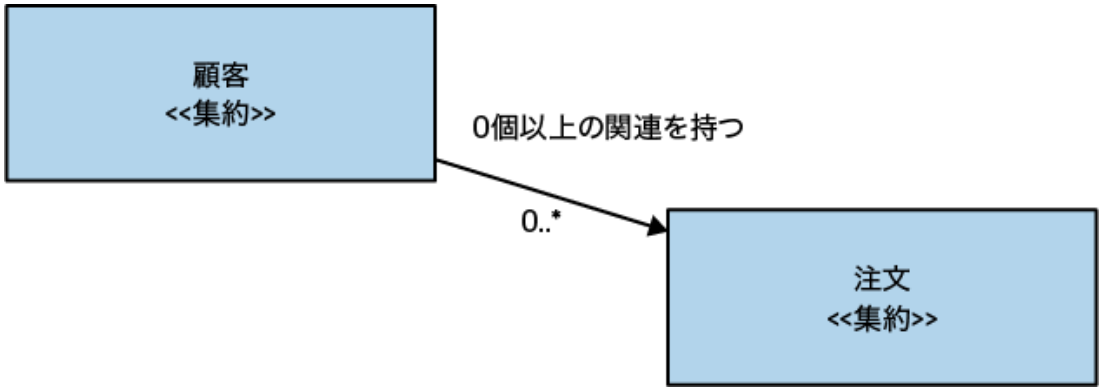

図1-18　1つの顧客集約は0個以上の注文集約と関連している

システムを集約に分割する方法はたくさんあるものの、そのいくつかはとても主観的だ。パフォーマンスの理由や実装のしやすさから、後々で集約を作り直す場合もある。私の場合は、最初のうちは実装の関心事は二の次とし、他の要因が現れるまではシステム利用者のメンタルモデルを初期設計の指針としている。2章では、開発者以外の仕事仲間の助けを借りてこうしたドメインモデルを形成するための共同作業として、イベントストーミングを紹介する。

1.4.2　境界づけられたコンテキスト

境界づけられたコンテキスト（Bounded Context）は、集約よりも大きな単位の組織境界を表す。その境界内では、明確な責任が果たされる必要がある。境界づけられたコンテキストは若干奥が深い。そのため、もう1つ、具体例で見てみることにしよう。

Music Corpで活動の中心となるのは、倉庫だ。出荷されている注文（や変則的な返品）を管理したり、新しい在庫の配送を行ったり、運搬用のフォークリフトを持っていたりする。経理部門もまた、活動の中心だ。経理は、給与計算の処理や出荷の支払いといった、あまり楽しい仕事ではないものの組織内の重要な機能を担っている。

境界づけられたコンテキストは、実装の詳細を隠蔽し、内的な関心事を持つ。たとえば、使用されるフォークリフトの種類は、倉庫にいる人以外にはほとんど関心がないことだろう。こうした内的な関心事は、外界から隠されるべきだ。外の世界はそうしたことを知る必要はないし、気にするべきでもない。

実装の観点から見ると、境界づけられたコンテキストには1つ以上の集約が含まれている。一部の集約は境界づけられたコンテキストの外部に公開され、他の集約は内部に隠される。集約と同様に、境界づけられたコンテキストは他の境界づけられたコンテキストと関連を持つことがある。サービスにマッピングされると、これらの依存関係はサービス間の依存関係になる。

1.4.3　集約と境界づけられたコンテキストをマイクロサービスにマッピングする

集約と境界づけられたコンテキストは共に、より広いシステムとの適切に定義されたインターフェイスを持つ、ひとまとまりの単位を提供してくれる。集約はシステム内のあるドメイン概念に焦点を当てた自己完結型の状態マシンであ

り、境界づけられたコンテキストは関連する集約のコレクションを表し、それらはより広い世界への明確なインターフェイスを備えている。

したがって、どちらもサービス境界としてうまく機能する。既に述べてきたように、マイクロサービス化を始めた段階では、扱うサービスの数を減らしたいはずだ。その結果、おそらくは境界づけられたコンテキストの範囲をサービスの対象とする必要があるだろう。環境に慣れ、それらのサービスをより小さなサービスに分割することに決めたら、集約を境界としてサービスを分割することに目を向けよう。

境界づけられたコンテキストをモデル化したサービスを後でより小さなサービスに分割すると決めたとしても、そのことを外部から隠せるというのが、この方法のポイントだ。たとえば、コンシューマーに粒度の粗いAPIを提示しておくことで、この決定を外部から隠蔽できる。サービスをより小さな部分に分解するという決定は、おそらく実装上の決定なので、できることなら隠したほうがいいだろう。

1.4.4　参考文献

ドメイン駆動設計の徹底的な探求は価値のある活動だが、本書の範囲からは外れる。もしこれをさらに追求したいのであれば、原典である『エリック・エヴァンスのドメイン駆動設計』（翔泳社）[8]か、Vaughn Vernonによる“Domain-Driven Design Distilled”（Addison-Wesley）[34]を読むことをお勧めする。

1.5　この章のまとめ

この章で議論したように、マイクロサービスは、ビジネスドメインに基づいてモデル化された独立したデプロイ可能なサービスだ。それらはネットワークを介して相互に通信する。情報隠蔽の原理をドメイン駆動設計と共に利用して、独立して作業しやすい安定した境界を持つサービスを作り、さまざまな形態の結合を減らすためにできることをしていく。

また、マイクロサービスアーキテクチャがどこから来たのかの簡単な歴史をざっと説明し、その基盤となる膨大な先行研究のさわりに触れた。また、マイクロサービスアーキテクチャに関連するいくつかの課題についても簡単に確認した。次の章では、マイクロサービスアーキテクチャへの移行をどのように計画

するかを説明することに加え、マイクロサービスアーキテクチャが自分たちに適しているかを判断するのに役立つ手引きを提供する。